Week 10: Embeddings and RAG

AI Development Course

Search by meaning, and answers from your own documents

Tested with google-genai 2.30.0 and gemini-embedding-001, anthropic 1.13.0 and claude-haiku-5-5, and the deal screener's offline tests, 13 October 2026. Real Google calls, AI Studio, school accounts, a signed-in session, GitHub, and Mac and Windows laptops are being tested.

Today

- Keyword search vs search by meaning
- Embeddings: text as a list of numbers
- A Google key, and what the free tier does with your text
- RAG: chunk, index, retrieve, answer
- Grounding: "My notes don't say."
- RAG over your own course-safe documents
- Review, Docker, pull request

Keyword search misses things

Your note:

> The association dues on the Riverside condo are $185 a month.

Your search: **"HOA fees"**

- Keyword search: **nothing**. No shared words.
- A person: obviously the same thing.
- Search by meaning: finds it.

Embeddings: meaning as numbers

- An embedding model turns text into a long list of numbers
- `gemini-embedding-001`: 3,072 by default; the course keeps **768**
- Similar meaning, similar numbers, even with different words
- **Cosine similarity**: how alike two meanings are, from -1 to 1

Google: "Cosine similarity is a good distance metric because it focuses on direction rather than magnitude."

Your first embeddings

Pair	Expected
Dues note vs "How much are the HOA fees?"	**higher**
Dues note vs contractor's phone note	lower
HOA question vs contractor's phone note	lower

`embed.py` in `~/first-call` prints all three (being tested: the real scores).

A Google key (before class)

1. https://aistudio.google.com/apikey > **Create API key**
2. School account first; if "you do not have access", a personal account (18+)
3. One line in `.env`, no quotes: `GEMINI_API_KEY=AIza...`
4. **Don't** also set `GOOGLE_API_KEY`: it wins
5. Leaked? Delete it in AI Studio **first**

Same rules as week 9: you type it, git ignores it, Claude Code can't read it.

The free tier: what Google does with your text

Google's terms, for the free ("Unpaid") services:

> "human reviewers may read, annotate, and process your API input and output."

> "Do not submit sensitive, confidential, or personal information to the Unpaid Services."

So: **only send documents you'd let a stranger read.**

RAG in four steps

1. **Chunk:** split documents into small pieces
2. **Index:** embed every piece **once**, save to a file
3. **Retrieve:** embed the question, pick the 3 closest pieces
4. **Answer:** Claude answers **only** from those pieces, and names them

The index holds your documents' text: git and Docker ignore it.

The deal screener's notes

`notes/` : 8 made-up investor notes

```
python3 -m dealscreener.notes index
python3 -m dealscreener.notes ask "What are the HOA fees like in Riverside?"
```

- `Indexed 9 chunks from 8 notes.` (being tested)
- `Notes that match best:` the Riverside dues note first
- The answer, `Sources: ...`, and the Claude cost

Grounding: the instructions

```
You answer questions about a property investor's own notes.
Answer only from the notes below. Do not use anything else you know.
After your answer, list the file names you used, after the word Sources:.
If the notes do not answer the question, reply exactly: My notes don't say.
```

Without it, the model answers from general knowledge, just as confidently.

A question the notes can't answer

```
python3 -m dealscreener.notes ask "What did I note about solar panels?"
```

- Retrieval still returns 3 chunks: something is always "closest"
- None of them mentions solar panels
- Answer: `My notes don't say.` (being tested)

When things go wrong, it says so

What happened	What you see
Notes edited after `index`	"Your notes changed since the index was built. Run: ..."
No Google key	"No Google API key found. Put GEMINI_API_KEY in a .env file ..."
Free-tier limit	"Too many requests for the free tier right now. ..."
No Claude key	the matching notes, "... Here are the notes that match best instead."

Never a key, never a traceback.

Course-safe documents only

Good	Not allowed
Notes or guides **you** wrote	Real people's names, emails, addresses
Public material, with its source	Customer, employer or client documents
Made-up examples, labelled	Anything you'd mind a stranger reading

Your RAG feature

1. Branch `rag` first
2. `git check-ignore .env`, and the week 9 deny rule is still there
3. Add `GEMINI_API_KEY` to `.env` with a text editor
4. 5 to 15 course-safe documents in a folder
5. `/superpowers:writing-plans` with the guardrails, then `subagent-driven-development`
6. Ask three questions, one your documents can't answer

Tests that don't call anyone

- A **fake embedder**: words to numbers, so "dues" and "HOA" still match
- A **fake model**: canned answers
- A test for "no Google key"
- One live test, only with `RUN_LIVE_TESTS=1`

Review, Docker, finish

- `safe-and-private`: keys, `.env`, the index, the documents, real calls in tests
- Docker: `--env-file .env`, share the documents folder (read-only) and the index file
- `docker run --rm <image> ls -a`: no `.env`, no index
- Files changed: Find `sk-ant-` and `AIza`: only fakes
- Merge on GitHub, then pull

The assignment

Criterion	Points
Merged RAG PR: fake embedder and model, "no Google key" test, course-safe documents, no key or index	4
Three questions with retrieved documents and answers (one unanswerable), and the index screenshot	3
Reflection: chunk size, and grounding	3

Next week: tools, agents and MCP.